NFL Big Data Bowl 2026 - Prediction

Rishitha Voleti, Minal Arunashalam, Vaibhav Sonnakul, and Mohamed Shaik

Introduction

Our project is for the NFL Big Data Bowl 2026 Prediction competition. We're trying to predict where every player will move right after the quarterback throws the ball.

The competition gives us all the player tracking data (position, speed, etc.) up to the moment of the pass. We also get the targeted receiver and the ball's landing spot. We have to use that info to predict a sequence of (x, y) coordinates for every player for the entire time the ball is in the air. The official evaluation metric is Root Mean Squared Error (RMSE), so our model's accuracy will be judged on how close its predicted coordinates are to the real ones.

We'll start off by using methods from our slides and textbook. Player movement is complex and non-linear, so a simple linear model won't work. We'll start by building a basic Multi-Layer Perceptron (MLP) as a baseline. We'll train it with Backpropagation and use a Squared Loss function, which makes sense since it's directly related to the RMSE metric.

After testing it out with that, we plan to spend most of our time on more advanced models. Neural networks seem like a good fit because they can learn features from the raw tracking data. We have two main ideas. First, we'll try Convolutional Neural Networks (CNNs), likely a 1D-CNN, to process the features from each frame and find spatial patterns. Second, since we're predicting a sequence of coordinates, we'll use Recurrent Neural Networks (LSTMs), which are built for time-based data.

The hybrid CNN-LSTM (from our related works) approach seems promising, as other researchers have used it for a similar NFL trajectory problem. We've seen from other papers that just using raw coordinates isn't enough, so we'll experiment with creating features like player-to-player distances, relative angles, or changes in direction. Finally, we know from other research that these sports datasets can be massive, so we'll need to make sure our model runs efficiently. Most importantly, we need to ensure this model does not overfit on the training data. We'll use validation techniques like K-fold cross-validation and early stopping to make sure our model can be accurately deployed to the actual testing data, which (per the competition rules) will be from live NFL games.

Related Works

1. In the paper “Prediction of Football Match Results with Machine Learning” the authors Rodrigues and Pinto (2022) talked about their research and development of a machine learning model to predict the outcomes of football (soccer) matches using past match and player statistics. The authors tested various algorithms, mainly classification models to categorize outcomes into wins, draws, or losses. They showed that certain features like player statistics, team performance metrics, and contextual information played a big role in improving prediction accuracy. It even led to increased profit margins when testing betting simulations. This work is mainly one Europe focused professional soccer the methodology used can be relevant to our predictive modeling with American football. Their approach to feature selection and model evaluation gives us a foundation to explore NFL player movement outcomes.

2. Cheong, L. L., Zeng, X., & Tyagi, A. (2021) directly focuses on trajectory prediction in the NFL using actual player tracking data. can be used to predict future defensive player trajectories using a CNN-LSTM model. They make a CNN-LSTM hybrid model, which demonstrates how CNNs can be applied to extract spatial features from each frame of player positions (like x, y coordinates), while LSTM layers capture temporal dependencies across multiple frames. From this, we can learn how to structure input data as sequences of spatiotemporal features

and how to use deep learning models to handle both the spatial layout of players on the field and their movement patterns over time. The paper also highlights the importance of engineering good features like relative distance between players and framewise directional changes, which improve the model's ability to capture dynamics between players. This paper provides a very relevant example of how deep learning can be used to model/predict movement of multiple positions at a time, which is very useful for our project, since we are trying to predict player movement as well.

3. Getting accurate and efficient prediction models in sports prediction is a big challenge these days. Traditionally, predicting students' sports performance relied on teachers' personal judgment and basic statistical methods, which resulted in low accuracy and didn't really meet what was needed in practice. Different machine learning approaches have been tried to fix this, like K-Nearest Neighbor (KNN), Support Vector Machines (SVM), Logistic Regression, and Neural Networks. However, these traditional models often have trouble with complex relationships in sports data, like how the number of shots, free kicks, and corner kicks relate to the actual match results. Also, many existing prediction models struggle with class imbalance problems. This is when certain performance categories have way more samples than others, which makes the model biased toward the majority class during training. To fix these problems, Wang and Ren (2024) proposed an updated U-Net Convolutional Neural Network model for sports achievement prediction that has several improvements. First, the model uses dense connections which allows each layer to directly access features from all previous layers. Second, the model adds an attention module in the feature extraction stage, which helps the network focus on the most important features while ignoring irrelevant ones. Third, an improved mixed loss function is used to handle class imbalance by giving different weights to different classes, making the model pay more attention to minority classes during training. Their experimental results showed that the improved U-Net achieved 93% accuracy in sports performance prediction, which is 4.22% better than the original U-Net and 5.22% better than DUNet. These improvements show that combining dense connections with attention mechanisms can effectively improve both feature extraction and prediction accuracy. While their project is focused on classification and class imbalance, the success of their attention module is very relevant to our project. It suggests that adding a similar attention mechanism to our own (yet to be achieved) CNN-LSTM model could be a key way to improve its accuracy, helping it focus on the most critical player-to-player interactions on the field and ignore less important data.

About our Dataset

The 2026 NFL Big Data Bowl dataset comes from NFL Next Gen Stats tracking during pass plays. Each play splits into two parts: input tracking before the throw and output tracking after release. Input files contain frame by frame info for every player: position (x, y), speed (s), acceleration (a), direction (dir), orientation (o), plus context like play direction, player position, team side, and ball landing spot (ball_land_x, ball_land_y). Each row has a unique ID from game_id, play_id, nfl_id, and frame_id.

Output files provide the ground truth future positions we need to predict. For each relevant player and frame after the throw (up to num_frames_output), the output has true x and y coordinates. These align with the same IDs so they can be joined to input data. Input files answer "Where was everyone right before the throw?" Output files answer "Where did this player move while the ball was in the air?" The player_to_predict flag marks which players' trajectories get scored.

We built the training set by merging input and output CSVs on game_id, play_id, and nfl_id. This gives us one dataset where input features (x_input, y_input, s, a, dir, o) align with target positions (x_output, y_output). Every row represents one player at one input frame paired with their true next location.

Feature Engineering

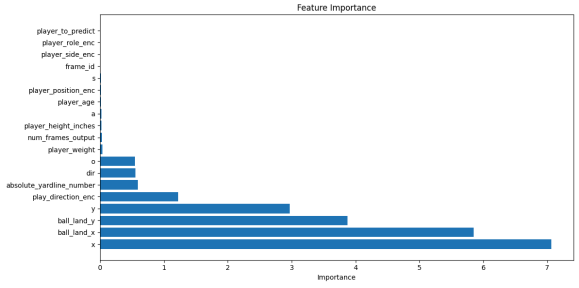
Initial experiments used 6 basic features (x, y, speed,

acceleration, direction, orientation), but player movement needs richer context. We expanded to 19 features capturing both individual state and field context. These include the original 6 plus ball_land_x, ball_land_y, play_direction_enc, player_position_enc, player_side_enc, player_to_predict, frame_id, and physical attributes like height, weight, age. We also computed absolute_yardline_number to standardize field position regardless of play direction.

Before training, we standardized features using scikit-learn (StandardScaler) so all variables are on the same scale. This matters because positional values (yards), angles (degrees), and acceleration operate on different ranges. For CNN and CNN-LSTM models, we reshaped inputs to the 1D CNN format: (X examples, 1 channel, N features).

We split the data 80/20 for training and testing. The test set contains actual target values, and all models are tested on this set for final metrics.

Feature importance analysis showed current position (x, y) had highest importance, which makes sense since current location predicts next location best. Ball landing coordinates ranked high too, showing players react to where the ball goes. Play direction had moderate importance, suggesting players adjust based on field direction. Player attributes like age, height, weight had minimal impact on short-term prediction.



Model Architecture

We tested multiple models: linear regression, multilayer perceptron (MLP), 1D convolutional neural network (1D-CNN), and CNN-LSTM. Linear regression and MLP were baselines. We tested 1D CNN alone first, then added LSTM to build the final CNN-LSTM model. We expected CNN-LSTM to perform best based on Cheong et al. (2021). Since trajectories depend on both position/field state at throw start and how that state changes over time, we designed models around spatial features and temporal patterns. All models use the same input features, and the target is always the next-frame position (x, y).

Linear Regression Model

As an initial baseline, we built linear regression to predict future positions. Using tracking data from several weeks, we merged each player's current state with their future coordinates to build a labeled training set. The goal wasn't to build the best model, but to create a simple reference that captures basic movement and gives a clear benchmark for complex models later.

RMSE directly measures average position error in yards, so it's easy to interpret in football context: RMSE ≈ 5.62 yards means predicted future position is about 5-6 yards from true location on average. RMSE is natural for our task since the target is a coordinate and we care about distance error. Using both train and test RMSE lets us see if the model overfits training data or maintains similar error on unseen plays.

R^2 shows how much variation in future positions the simple linear model explains. Train R^2 ≈ 0.81 and Test R^2 ≈ 0.81 mean about 81% of variability in player locations can be captured using only current position, speed, acceleration, direction, and orientation. R^2 measures how well regression fits data by showing the proportion of dependent variable variance that's predictable. It shows even a simple model has substantial power but leaves room for improvement. R^2 is for linear regression models, so we won't use it for neural networks like CNN, but it was useful here.

We also recorded training and prediction times to measure efficiency. This confirmed linear regression is extremely fast computationally, which is expected.

The efficiency makes it a good reference for comparing accuracy and cost of advanced models.

Multi-Layer Perceptron Model

For our next model we tested a MLP model to better capture nonlinear relationships between features and movement. Player movement is often influenced by complex interactions like speed, acceleration, and direction shifts. MLP is better suited for these nonlinear patterns.

We used the same merged dataset where each sample has current state paired with future coordinates. This ensures performance differences come from architecture, not data processing.

The MLP has multiple fully connected layers with nonlinear activation functions that build progressively better movement representations. We standardized all input features before training.

The MLP achieved a test RMSE of 6.32 yards (with 19 features). Compared to linear regression, the MLP showed worse performance, indicating nonlinear modeling contributed to less accurate understanding of movement.

We also computed test R^2 to quantify how much variance in future positions the model explained. The MLP achieved R^2 of 0.87 compared to linear regression, showing MLP models a larger portion of underlying movement variability.

Looking at the training history graph, both train and validation loss decrease over epochs, which is what we want. Training loss drops sharply in the first few epochs then stabilizes around epoch 15, settling at approximately 15. Validation loss follows a similar pattern but remains slightly higher, plateauing around 50. This gap suggests some overfitting, but validation loss doesn't increase over time, which would be a red flag. The model seemed to be extremely overfitting.

1D Convolutional Neural Network Model

After establishing baselines, we moved to a 1D CNN to see if spatial feature extraction could improve predictions. CNNs are good at finding local patterns in data, and we thought they could learn relationships between nearby features (like how speed and acceleration interact with position and direction).

The 1D-CNN architecture uses multiple convolutional layers followed by pooling to extract hierarchical features from input. We reshaped input data from (samples, features) to (samples, 1 channel, features) to match CNN input format. The model learns filters that detect patterns across the feature dimension.

The 1D-CNN achieved impressive results with test RMSE of 3.98 yards and standard deviations of 3.60 yards for X error and 3.34 yards for Y error. This is a substantial improvement over linear regression (5.62 yards) and MLP (6.32 yards). Training history shows both losses decrease smoothly and converge well, with validation loss stabilizing around 12.5 and training loss around 18.5. The small gap between training and validation loss shows the model isn't overfitting significantly.

Prediction scatter plots for both X and Y coordinates show tight clustering around the diagonal line (perfect predictions), indicating accurate predictions across the full range of field positions. Error distribution histograms are centered near zero with narrow spreads, which is exactly what we want. The Euclidean error distribution has a mean of 3.98 yards, showing the CNN successfully learned meaningful spatial patterns from player tracking features.

CNN-LSTM Hybrid Model

Our final model combines CNN's spatial feature extraction with LSTM's temporal sequence modeling. The architecture applies convolutional layers to extract spatial patterns from features, then feeds CNN output into LSTM layers that learn how patterns evolve over time. This hybrid approach was inspired

by Cheong et al. (2021) on NFL defensive trajectory prediction.

The CNN-LSTM achieved the best performance of all models, with a test RMSE of 3.75 yards. X error standard deviation was 3.34 yards and Y error standard deviation was 3.12 yards, both slightly better than 1D-CNN alone. This improvement, while modest, suggests LSTM layers successfully capture some temporal dynamics the CNN alone cannot model.

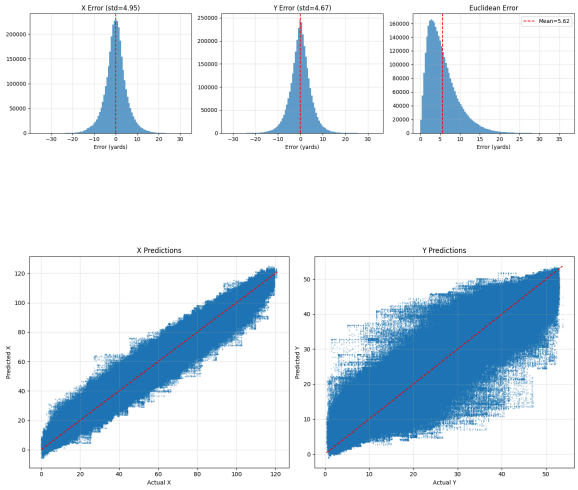
Training history for CNN-LSTM shows a similar pattern to 1D-CNN, with smooth convergence of both losses. Validation loss stabilizes around 10.5 and training loss around 18.5. The small gap shows good generalization. Prediction scatter plots show even tighter clustering around the perfect prediction line compared to 1D-CNN, especially for Y coordinates. Error distributions are well-centered with slightly narrower spreads, confirming the LSTM component provides incremental benefit.

Results & Analysis

As we moved from simple baselines to more advanced deep learned models, there is a constant improvement in the model performance. For all the models, we evaluated using RMSE as it measures the average positional error in yards and is required for the competition.

Linear Regression

Using linear regressions it gave us an RMSE score of 5.62 yards. This tells us that without being able to model nonlinear relationships, player movement can still be predicted and explained by basic kinematic features like position, speed, direction, acceleration and orientation. We can confirm this also with the relatively high R squared value of approximately 0.81. The error distribution graphs below show a centered distribution which means it produces consistent prediction without a strong bias. It also indicates that it lacks the precision needed to accurately model complex movement patterns.



The scatter plot above highlights the limitations of linear regression. The model struggles to adapt to sudden changes in directions and nonlinear movement patterns caused by player reaction. As the complexity of the model increases, we can see more optimal results.

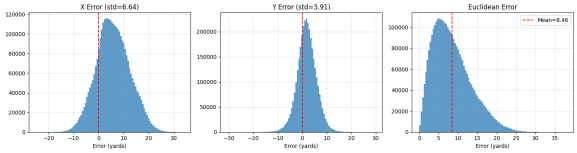
MLP

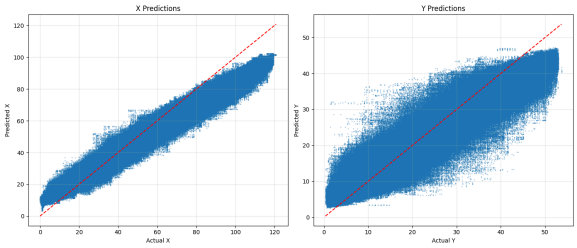
The Multi Layer Perception was used as a baseline model to use nonlinear data and predict this data. In theory, the MLP should outperform the linear regression as it can model complex interaction between speed, direction, acceleration and contextual variables.

In our results, MLP achieved a RMSE Score of approximately 6.32 yards which is astonishing as it performed worse than Linear Regression. While the model achieved a higher R squared value of around 0.87, this improvement did mean that it had better positional accuracy. This discrepancy highlights the difference between explaining variance and minimizing spatial error.

Below the training plots show a sharp decrease in the training loss during early epochs, which was followed by stabilization. In contrast, the validation loss remains significantly higher and plateaus early. This gap suggests overfitting, where the model learns patterns specific to the training data but fails to generalize to unseen plays.

The error distribution plots reinforce the conclusion. Compared to linear regression, the MLP exhibits wider error distributions and more dispersed scatter plots. These visuals indicate that fully connected architectures struggle to learn meaningful structure from player tracking data when spatial relationships are not explicitly encoded.

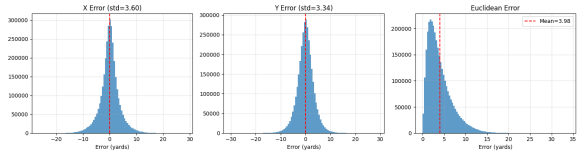


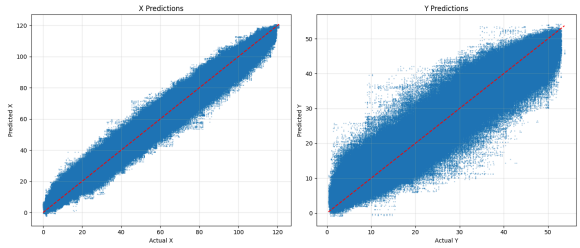


1D-CNN

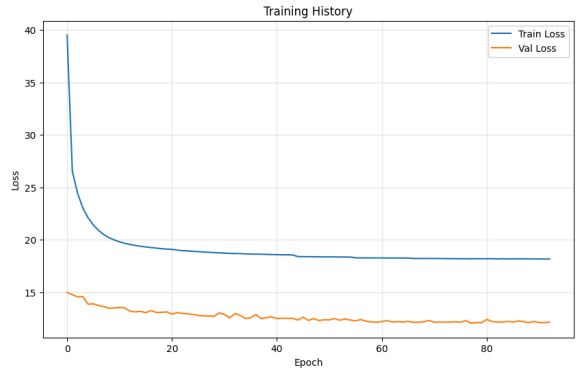
The on dimensional Convolutional Neural Network represents a major improvement over both baseline models. The CNN achieved a test RMSE of approximately 3.98 yards, which is a substantial reduction in average positional error. Thai improvement corresponds to nearly a two yard gain over the linear regression baseline, which is a significant improvement.

The scattered plot below shows tight clustering around the diagonal line for both x and y positions. This indicates that the CNN performs well across the full range of field positions. Additionally, the error distribution histograms are centered close to zero with narrower spreads, showing that more predictions fall within a small distance of the location.





The training and validation loss curves show smooth convergence with a relatively small gap, indicating good generalization. Unlike MLP, CNN does not exhibit strong overfitting behavior, This suggests that convolutional layers successfully extract reusable spatial patterns across features.



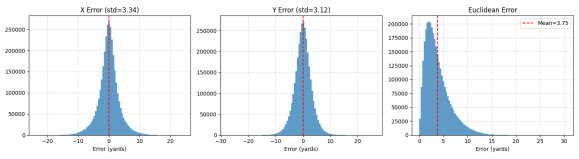
These visualizations are important because they provide qualitative confirmation of the numerical metrics. Together, they demonstrate that modeling

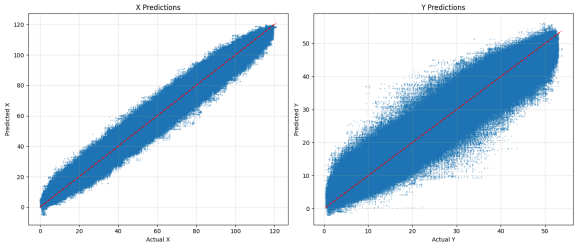
spatial relationships between features significantly improves prediction accuracy.

CNN-LSTM

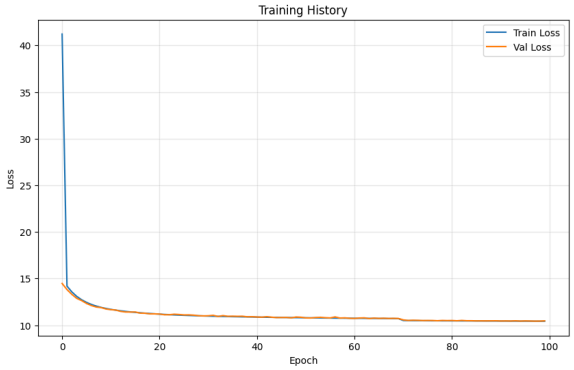
The CNN LSTM hybrid model achieved the best overall performance with a test RMSE of approximately 3.75 yards. Although the improvements over the CNN alone is modest, it is consistent across metrics and visualizations. This result suggests that temporal modeling added meaningful information beyond status spatial features.

The error distribution plots further confirm this improvement. Compared to the CNN, the CNN-LSTM model exhibits slightly narrower distributions, indicating more consistent predictions. These visuals support the conclusion that LSTM layers help capture short term temporal dependencies in player movement during the ball flight window.





The training history plot shows stable convergence with validation loss remaining close to training loss. Thai indicated strong generalization and minimal overfitting. THe scatter plot shows even tighter clustering around the ideal prediction line compared to the CNN, especially for y coordinate predictions.



Overall the figures included in the results section play a crucial role in interpreting model behavior. While RMSE provides a single summary metric, the scatter loss, loss curves, and error histograms reveal how models learn, fail and generalize, This helped us understand the progressions in performance.

Conclusion

In this project we explored different machine learning approaches for predicting NFL player trajectories during a play. We first started with simple baselines and then progressed into more advanced deep learning model architectures. We showed clear performance gains as the models became better at capturing spatial and temporal structure in player movement. Linear regression provided a strong and interpretable baseline, while MLP struggled to generalize due to overfitting and lack of explicit spatial modeling. In contrast convolutional models proved highly effective with 1D-CNN significantly reducing prediction error by learning meaningful spatial relationships among features. The CNN-LSTM achieved the best overall performance confirming that incorporating temporal dynamics yields additional improvements.

Although the CNN-LSTM model achieved our best results, its performance still falls short of the top score on the official competition leader hoard. This gap highlights the difficulty of the problem and the complexity of real player interaction in live NFL games. Several limitations contributed to the gap from us to the top of the competition. One major constraint was hardware availability. Training more

complex models such as deeper CNN-LSTM stacks or transformer models was not feasible given our computational resources. These models often require significant GPU memory and longer training times.

Additionally our approach focused more on short term trajectory prediction without modeling play intent or path planning. Given more time, path planning would have been incorporated based on the player's role and their historical movement behavior.

Modeling how players choose paths relative to the game could help capture more complex interaction that simple coordinate based prediction cannot fully represent.

Despite these limitations, this project demonstrates that deep learning models can meaningfully improve player trajectory prediction when spatial and temporal structure are properly modeled. The results provide a strong foundation for future work and offer valuable insight into how player tracking data can be used to better understand and predict movement in football.

Team Member Contribution

Each group member contributed equally to the project. Mohamed worked on the linear regression model. Rishitha worked on the MLP model. Vaibhav worked on the 1D-CNN model. Minal worked on the CNN-LSTM. We all worked on identifying and engineering the right features, and optimizing model performance together. After our models were trained and tested we all equally contributed to various parts of the report.

Citations

1. Rodrigues, F., & Pinto, Â. (2022). Prediction of football match results with Machine Learning. Procedia Computer Science, 204, 463–470. https://doi.org/10.1016/j.procs.2022.08.057

2. Cheong, L. L., Zeng, X., & Tyagi, A. (2021). Prediction of defensive player trajectories in NFL games with defender CNN-LSTM model. Amazon Science.https://www.amazon.science/publications/prediction-of-defensive-player-trajectories-in-nfl-games-with-defender-cnn-lstm-model

3. Wang, G., & Ren, T. (2024). Design of sports achievement prediction system based on U-net convolutional neural network in the context of machine learning. *Heliyon*, *10*(10), e30055. https://doi.org/10.1016/j.heliyon.2024.e30055

4. Michael Lopez, Tom Bliss, Ally Blake, Yao Yan, Martyna Plomecka, and Addison Howard. NFL Big Data Bowl 2026 - Prediction. https://kaggle.com/competitions/nfl-big-data-bowl-2026-prediction, 2025. Kaggle.